Introduction

Every so often a two-line snippet of Python code exposes a gap in how most developers think their language works. This is one of those cases. It looks like a beginner question — set a key, set another key, print the dict — but it quietly tests whether you understand the difference between *value equality* and *object identity*, and how Python's dictionary actually decides whether two keys are "the same."

Interviewers love this kind of question because it can't be memorized away. You either understand hashing and equality, or you guess. And guessing wrong on something this fundamental is a strong signal that a candidate hasn't looked under the hood of the data structure they use every single day. If you've ever been surprised by a dict behaving "wrong," or you want to be the person in the room who explains *why* instead of just *what*, this one is worth five minutes of your time.

Problem Overview

Here's the setup, rewritten plainly: you create an empty dictionary. You insert the integer `1` as a key, mapped to the string `"one"`. Then you insert the boolean `True` as a key, mapped to the string `"yes"`. Finally, you print the dictionary.

The question is simple: does the dictionary end up with two entries, or one? And if it's one, which value survives?

At first glance, `1` and `True` look like completely different objects — one is a number, the other is a boolean flag. Most people expect the dictionary to hold both, since they're written differently and represent different concepts in everyday code. That instinct is wrong, and understanding *why* it's wrong is the entire point of this exercise.

Example

```
d = {}
d[1] = "one"
d[True] = "yes"
print(d)
```

Output:

```
{1: 'yes'}
```

Only one key survives: `1`, but with the value `"yes"`. The `"one"` value is gone, overwritten. There is no separate entry for `True`. If you were expecting `{1: 'one', True: 'yes'}`, you've just met one of Python's most instructive quirks.

Intuition

The key insight is that a Python dictionary never asks "are these the same *type*?" It asks two much narrower questions: "do these two keys hash to the same value?" and "are these two keys equal?" If both answers are yes, the dictionary treats them as the *same key*, full stop — regardless of what type they are or what they look like in your code.

In Python, `bool` is a subclass of `int`. This isn't a coincidence or an implementation detail buried deep in CPython — it's a deliberate design choice baked into the language, dating back to when `bool` was added in Python 2.3. Because of that inheritance, `True` behaves as `1` and `False` behaves as `0` almost everywhere numbers are expected. That's why `True == 1` evaluates to `True`, and why `hash(True) == hash(1)`.

When you write `d[1] = "one"`, Python computes `hash(1)`, finds (or creates) a bucket for it, and stores the pair. When you then write `d[True] = "yes"`, Python computes `hash(True)`, which is identical to `hash(1)`. It looks in that same bucket, finds an existing key that is *equal* to `True` (because `1 == True`), and concludes: this isn't a new key, this is the same key, just written differently. So instead of inserting a second entry, it updates the value associated with the existing key.

An experienced engineer doesn't memorize this fact in isolation — they arrive at it by remembering the general rule: **dictionary key identity is governed by `__hash__` and `__eq__`, not by type or how the literal was spelled in the source.** Once you internalize that rule, `1` and `True` colliding stops being a surprise and becomes an obvious consequence.

Brute Force Solution

There isn't really a "brute force" way to solve this — it's not an algorithmic problem, it's a comprehension check. But if you wanted to *verify* the behavior programmatically instead of reasoning about it, you could write a small brute-force checker that inserts a batch of candidate keys and inspects the resulting dictionary size.

Idea: Insert every key from a list one at a time, and after each insertion, record the size of the dictionary and whether it changed.

```
def track_collisions(pairs):
    d = {}
    history = []
    for key, value in pairs:
        before = len(d)
        d[key] = value
        after = len(d)
        history.append((key, value, before == after))
    return d, history
```

Advantages: Simple, makes every collision visible step by step, useful for debugging or teaching.

Disadvantages: Doesn't explain *why* a collision happened — you still need to reason about hash and equality separately. It only observes symptoms.

Time complexity: O(n) for n insertions, since dict insertion is O(1) average case. **Space complexity:** O(n) for the dictionary plus the history list.

Optimal Solution

The "optimal" solution here isn't a faster algorithm — it's the correct mental model, applied directly. Rather than testing empirically, you reason from Python's actual key-matching rule:

Step	Key inserted	`hash(key)`	Existing key with same hash?	Equal to it?	Result
1	`1`	`hash(1)`	No	—	New entry: `{1: 'one'}`
2	`True`	`hash(True)` (== `hash(1)`)	Yes, `1`	`True == 1` → Yes	Overwrite: `{1: 'yes'}`

Two conditions both had to hold for the overwrite to happen: matching hash *and* matching equality. If either had differed, you'd get two separate keys. This table is the whole solution — no code needed to "solve" it, just correct tracing of the rule.

It's also worth noting the reverse isn't automatically true: two objects can hash the same (a hash collision) without being equal — that's a *real* hash collision, handled internally by open addressing or chaining, and it never causes an overwrite. `1` and `True` are not a hash collision in that sense; they are a case of true equality between different-looking objects.

Python Code

```
def demonstrate_key_collision() -> dict:
    """Show that True and 1 map to the same dictionary key."""
    d = {}
    d[1] = "one"
    d[True] = "yes"
    return d


if __name__ == "__main__":
    result = demonstrate_key_collision()
    print(result)                       # {1: 'yes'}
    assert result == {1: "yes"}
    assert len(result) == 1
    assert hash(1) == hash(True)
    assert 1 == True
    print("all checks passed")
```

Code Walkthrough

- `d = {}` creates an empty dictionary — no keys, no hash table entries yet.
- `d[1] = "one"` computes `hash(1)`, places it in a bucket, and stores `"one"` as the associated value.
- `d[True] = "yes"` computes `hash(True)`. Since `bool` subclasses `int`, this hash equals `hash(1)`. Python checks the bucket, finds `1` already there, confirms `1 == True`, and overwrites the value in place rather than adding a new slot.
- `print(result)` shows the dictionary has exactly one key, `1`, holding the most recently assigned value, `"yes"`.
- The `assert` lines in `__main__` act as a self-check: they confirm the collision happened, the dict has one entry, and the underlying reason (equal hash, equal value) holds.

Dry Run

Start: `d = {}`

1. `d[1] = "one"` → hash bucket for `hash(1)` is empty → insert `(1, "one")` → `d = {1: 'one'}`
2. `d[True] = "yes"` → compute `hash(True)` → identical to `hash(1)` → look in that bucket → find key `1` → check `1 == True` → `True` → don't create a new slot, update value → `d = {1: 'yes'}`

3. `print(d)` → `{1: 'yes'}`

Final state: one key, one value, and `"one"` is gone for good — it was never stored anywhere else.

Complexity Analysis

- **Time:** O(1) average case for each insertion, since dictionary lookups and writes rely on hashing. Total for two insertions: O(1).
- **Space:** O(1) — the dictionary holds exactly one entry throughout, regardless of how many "equal" keys you insert.

These are correct because the number of *distinct* keys, not the number of insertion attempts, determines the dictionary's footprint. Inserting the same logical key a hundred times still results in one entry.

Alternative Solutions

You could sidestep the collision entirely by using a **composite key** that includes type information, such as `(type(key), key)`:

```
d = {}
d[(type(1), 1)] = "one"
d[(type(True), True)] = "yes"
print(d)  # {(<class 'int'>, 1): 'one', (<class 'bool'>, True): 'yes'}
```

This preserves both entries because the tuples are no longer equal — `(int, 1) != (bool, True)` even though `1 == True`. It's a reasonable pattern when you genuinely need to distinguish `1` from `True` as keys (for example, when parsing untyped data), but it adds overhead and complexity you shouldn't reach for unless the ambiguity actually matters in your program.

Edge Cases

- **`0` and `False`:** Same behavior — `hash(0) == hash(False)` and `0 == False`, so they collide exactly like `1` and `True`.
- **Empty dict:** No collision possible; the second insertion becomes the first and only entry, same as any single insertion.
- **Multiple overwrites:** `d[1] = "a"; d[True] = "b"; d[1.0] = "c"` — even `1.0` collides, since `hash(1.0) == hash(1)` and `1.0 == 1`. The dict ends up with `{1: 'c'}`.

- **Non-hashable keys:** Attempting `d[[1,2]] = "x"` raises `TypeError`, unrelated to this collision but a common follow-up trap.
- **Custom objects:** If a class overrides `__eq__` without overriding `__hash__`, instances become unhashable by default, sidestepping this issue but introducing a different one.

Common Mistakes

1. Assuming type equality is required for keys to collide — Python only checks hash and `__eq__`, not `type()`.
2. Forgetting that `bool` is a subclass of `int`, so `True` / `False` behave numerically in far more places than dictionary keys (arithmetic, indexing, `sum()`).
3. Expecting insertion order or "most descriptive key" to win — the *last write* always wins, regardless of which key "looks" more meaningful.
4. Confusing this with a hash *collision* in the technical sense (two unequal objects sharing a hash bucket) rather than genuine equality.
5. Assuming `is` and `==` behave the same for dict key matching — dictionaries use `__eq__`, not identity, so even distinct objects that are merely equal will collide.
6. Not testing with `assert len(d) == expected` after bulk insertions, which would catch this kind of silent overwrite early.

Interview Questions

- Why does `bool` subclass `int` in Python, and what are the consequences?
- What two conditions must hold for two dictionary keys to be treated as identical?
- How would you design a dictionary where `1` and `True` are distinct keys?
- What happens if a custom class defines `__eq__` but not `__hash__`?
- Can two unequal objects share the same hash value? What happens then?

Similar Problems

- **LeetCode 705 – Design HashSet:** builds intuition for how hashing and bucket collisions work under the hood.
- **LeetCode 706 – Design HashMap:** forces you to implement equality and collision handling explicitly, reinforcing this exact concept.
- **LeetCode 1 – Two Sum:** relies on dictionary key lookups where understanding hash/equality semantics prevents subtle bugs with numeric types.

- **LeetCode 242 – Valid Anagram:** another classic case where dict key behavior under different but "equal" inputs matters.

Key Takeaways

Python dictionaries key on hash and equality, never on type or spelling. Because `bool` is a subclass of `int`, `True` and `1` (and `False` and `0`) are genuinely equal values, not just similar-looking ones — so they collide as the same dictionary key. The value from the most recent assignment always wins. This one fact explains a whole category of "why is my dict smaller than I expected" bugs in real code.

Watch the Video

For the full breakdown with visuals of exactly how the hash bucket lookup happens step by step, watch the video here: {LINK}

About the Series

This breakdown is part of **Fun with Learning Technology**, a series built around exactly these moments — the two-line snippets that look trivial but expose a real gap in how a language works under the hood. Every entry is designed to take something you already write daily and make you understand it, not just use it.

Call To Action

If this changed how you think about Python dictionaries, like the video on YouTube and subscribe for more of these breakdowns. For deeper, written walkthroughs like this one delivered straight to your inbox, subscribe to the Substack. Drop a comment with a Python gotcha that once caught you off guard — there's a good chance it becomes the next one of these.

Quick Review

See rows disappearing/merging when Python dict keys look distinct but collide?

Custom __hash__/__eq__ mismatch or mutable-key mutation — dict treats equal-hash+equal-value keys as one slot.

Time complexity of a dict lookup/insert under key collisions?

O(1) average even with collisions (open addressing probing); O(n) worst case if hash function is bad.

Space complexity of storing n items in a Python dict?

O(n) — one slot per entry plus load-factor overhead from resizing (~1/3 extra capacity).

Trickiest bug with dict keys?

Using a mutable object (list) as key raises TypeError; using a custom class without __hash__ falls back to id(), breaking intended equality.

Dict key collision vs hash table 'collision resolution' in DSA?

Same concept — Python's dict is a hash table; interview questions about chaining/open addressing map directly to dict internals.

Interview follow-up: why override both __eq__ and __hash__ together?

Equal objects must hash equally or dict/set lookups break silently — Python doesn't enforce this for you.
